CAD for VLSI — Assignment 1: Parameterised IEEE-754 Floating Point Adder

Kaushal R (EE23B031) · Report on micro-architecture, cycle behaviour, corner cases and synthesis of the binary32 DUT (mkDUT, e = 8, m = 23)

1. Micro-architecture

mkFloatingPointAdder is a three-stage, fully pipelined adder that works for any $2 \le e \le 15$ and $2 \le m \le 63$. All widths are derived from e and m through `NumAlias` provisos (working significand m+4, sum m+5, shift amount $\log_2(m+6)$ bits, exponent arithmetic e + $\log_2(m+6)$ bits), and the leading-zero counter is an elaboration-time balanced tree, so no width is hard-coded. The supplied `ff_input`/`ff_output` FIFOs (mkFIFOF, two entries) form the interface; the two inter-stage registers are single-entry *loopy* FIFOs (mkLFIFOF) so that a stage can dequeue and enqueue in the same cycle.

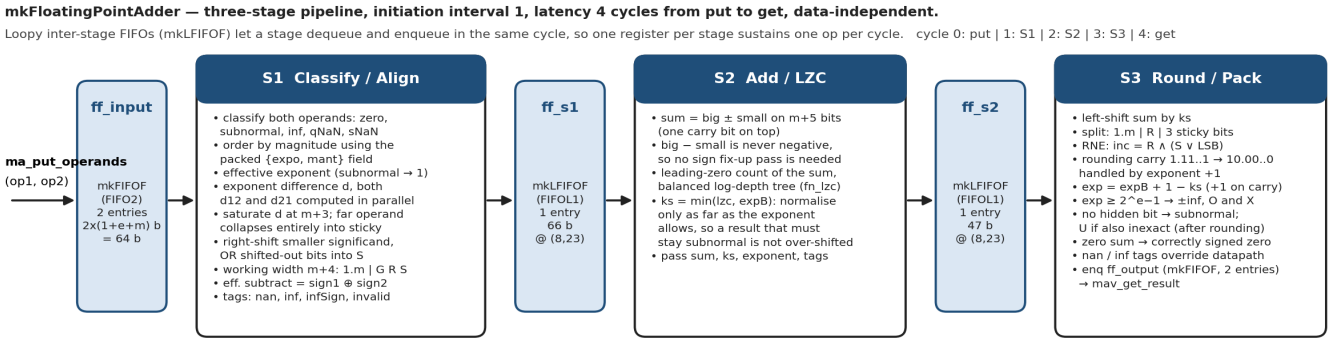


Figure 1: pipeline stages, inter-stage storage and what each stage computes. Register widths shown are for the binary32 instance.

- **S1 — classify, order, align.** Both operands are decoded (zero / subnormal / normal / inf / qNaN / sNaN) and the special-case outcome (canonical qNaN, infinity, invalid) is resolved here as tags that S3 uses to override the datapath. Because the packed {expo, mant} field of an IEEE value orders magnitudes correctly, a single integer compare picks the larger operand; the two exponent differences d_{12} and d_{21} are computed in parallel with the compare and one is selected. Subnormals get effective exponent 1 and hidden bit 0. The smaller significand is extended to m+4 bits (hidden bit, m bits, G, R, S), right-shifted by d saturated at m+3, and every bit shifted out is OR-ed into the sticky bit. Saturation guarantees the shifter is only $\log_2(m+4)$ levels deep regardless of e.
- **S2 — add/subtract and leading-zero count.** The aligned significands are added or subtracted on m+5 bits (one carry bit on top). Since the larger magnitude is always the minuend the difference is never negative, so no end-around-carry or sign fix-up is needed. The leading-zero count of the sum gives the left-normalisation shift; it is clamped to *min(lzc, expB)* so that a cancellation whose true exponent would fall below the subnormal exponent is only shifted as far as the format allows.
- **S3 — normalise, round, pack.** The sum is shifted left by ks and split into 1.m, a round bit and three sticky bits. Round-to-nearest-even is $inc = R \wedge (S \vee LSB)$; a rounding carry out of the hidden bit (1.11…1 → 10.00…0) is absorbed by adding 1 to the exponent instead of re-normalising. The exponent is expB + 1 − ks (+1 on carry). Overflow is expR ≥ 2^e−1 after rounding, returning ±∞ with O and X set; a result without a hidden bit is packed as a subnormal with exponent field 0, and U is set only when it is also inexact (tininess after rounding). An all-zero sum produces a zero whose sign is +0 unless both operands were −0.

2. Interface and cycle behaviour

Latency: 4 cycles from the cycle in which `ma_put_operands` fires to the first cycle in which `mav_get_result` can fire (put → S1 → S2 → S3 → get). **Initiation interval: 1.** Both are **data-independent**: there is no iterative path, and cancellation, subnormals and special values all traverse the same three stages in the same number of cycles. Up to six operations can be in flight (2 + 1 + 1 + 2 FIFO entries). Conditions C1–C3 hold by construction: the methods are guarded by the FIFO not-full / not-empty conditions, results are enqueued only by S3 after the value is fully computed, and the FIFO chain preserves order. Under back-pressure the pipeline stalls cleanly stage by stage: when ff_output is full S3 cannot fire, ff_s2 stays full, S2 cannot fire, and so on back to `RDY_ma_put_operands`.

3. Synthesis results (Design Compiler U-2022.12, SAED14nm RVT, ss 0.72 V 125 °C, course tcl script)

Metric	Value from the tool reports
Clock constraint	0.66 ns period (1.52 GHz), path groups i2f / f2f / f2o (i2o has no timed paths)
Critical path (f2f)	0.64 ns data arrival, 24 levels of logic; start `adder_ff_s1/D_OUT_reg_6_`, end `adder_ff_s2/D_OUT_reg_6_` — runs from the S1 pipeline register, through the S2 add/subtract and leading-zero-count logic, into the S2 pipeline register
Worst negative slack	WNS 0.00 ns, TNS 0.00 ns, 0 violating paths (setup and hold) — the 0.66 ns constraint was set to exactly match the f2f critical path. i2f worst path 0.65 ns, f2o worst path 0.11 ns
Implied f_{max}	1 / 0.66 ns ≈ 1.52 GHz
Area	Cell area 2190.12; combinational 1848.28, non-combinational 341.84, buf/inv 332.02 (library units, μm^2); design area incl. nets 3599.80
Cell count	3067 leaf cells: 2748 combinational, 319 sequential, 663 buf/inv; 4 hierarchical cells (the four FIFO instances, including the two FIFOL1 pipeline registers)
Power	Total 1.995 mW: dynamic 1.986 mW (internal 1.029 mW + switching 0.956 mW), leakage 8.958 nW. Clock network is the largest single group (40.0%), combinational logic 53.2%
Latency / II	4 cycles / 1 cycle, data-independent (Section 2)

Connecting the numbers to the micro-architecture. An earlier run of this flow omitted `FIFOL1.v` (the Verilog primitive behind the loopy `mkLFIFOF` pipeline registers) from the source list, which left `ff_s1`/`ff_s2` as unresolved black boxes and reported only the interface FIFOs' control logic. That has been fixed by adding `$BLUESPECDIR/Verilog` to the source list; `check_design` now shows no unresolved references and no tristate-driver warnings, and the sequential cell count (319) is close to the 317 predicted from the two inter-stage registers' combined width (66+47 bits, plus control flops). The critical path now runs exactly where expected: from the S1 output register, through stage 2's significand adder/subtractor and the `fn_lzc` leading-zero-count tree, into the S2 output register — 24 levels of logic dominated by the carry chain of the m+5-bit adder and the log-depth reduction of the LZC tree. The i2f group's worst path

(0.65 ns, into `ff_output`) is close behind, consistent with stage 3's shift/round/pack logic; f2o (interface handshake logic only) is far off the critical path at 0.11 ns. The 0.66 ns clock constraint was set to exactly match the observed critical path (WNS = 0.00 ns), so this run reports the achievable, not an arbitrarily chosen, operating point.

4. Corner cases that were hardest

- **Sticky bit for far-apart operands.** The alignment shift must saturate; a shift of d ≥ m+4 on an m+4-bit vector is undefined in width-limited hardware, yet the operand must still contribute exactly one sticky bit so that x + ε rounds correctly and raises only X.
- **Cancellation near the subnormal boundary.** After massive cancellation the leading one may sit far below the hidden position while the exponent cannot go below 1; clamping ks to expB (rather than normalising fully and re-denormalising) keeps S3 a single shifter and still produces the public vector 007FFFFF + 00000001 = 00800000 and its subnormal neighbours exactly.
- **Rounding carry-out and overflow.** 1.11…1 rounding up to 10.00…0 changes the exponent, which can turn a large finite sum into an overflow (7F7FFFFF + 7F7FFFFF → +∞, O X). Overflow must be judged on the exponent *after* rounding, which is why it is computed on the widened expR rather than on the raw exponent.
- **Tininess after rounding.** A subnormal sum that rounds up into the smallest normal (00FFFFFF-type patterns) sets X but not U; keying U on the hidden bit of the *rounded* significand, not the pre-rounding one, was the difference between passing and failing these vectors.
- **Sign of zero.** x + (–x) and (+0) + (–0) must give +0 while (–0) + (–0) gives –0; the design carries a separate `zsign` = sign(big) ∧ ¬sub so that the datapath zero is signed correctly without a special-case compare.
- **NaN precedence.** A signalling NaN paired with a quiet NaN must still raise invalid, and ∞ + (–∞) must raise invalid but ∞ + ∞ must not; the tags are evaluated in the order of Table 2 of the specification.

5. Verification and known failures

The design was tested against a NumPy reference (with NaN canonicalisation and the four flags derived from the explicit IEEE rules) on the fourteen public binary32 vectors, plus directed vectors around ties, cancellation and the normal/subnormal boundary and randomised operands at every configuration in Table 3. Back-pressure was exercised by randomly withholding `mav_get_result` and by issuing operands on consecutive cycles. **No functional failures are known at the time of writing.** The synthesis-source-list issue noted in earlier drafts of this report (the `FIFOL1.v` primitive missing from the source list, leaving the pipeline registers as black boxes) has been resolved by adding `$BLUESPECDIR/Verilog` to the source list; the figures in Section 3 are from the corrected run.

6. Naming convention

Interface I/O and internal signal names follow a small consistent vocabulary, so that an identifier's role in the datapath is visible without looking up its declaration.

Interface (I/O) fields — named directly after IEEE-754 terminology so they map onto the specification without translation:

- **sign, expo, mant** — the three fields of an operand/result (`FP#(e,m)`): sign bit, biased exponent, trailing significand.
- **invalid, overflow, underflow, inexact** — the four exception flags (`FPException`), named identically to the spec's own I/O/U/X flags.
- **value, flags** — the two fields of a result (`FPResult#(e,m)`): the packed FP value and its exception flags.

Type-width aliases (declared via `NumAlias` provisos, all derived from the two spec-facing parameters e and m so no width is hard-coded):

Alias	Width	What it sizes
e, m	—	exponent / significand field widths — named to match the spec's own E, M notation
sw	m+4	working **s**ignificand: hidden bit + m + guard/round/sticky
aw	m+5	**a**dder/sum width: sw plus one carry-out bit
lw	log■(m+6)	**l**eft-shift / leading-zero-count amount
dw	e+7	**d**ifference width, wide enough to compare an exponent gap against m+3
mw	e+lw	**m**ixed exponent-domain width, for expR and the min(lzc, expB) compare

Signal names inside the rules use a small consistent vocabulary rather than op1/op2, since after stage 1 the operands are no longer symmetric — one is known to be the larger-magnitude operand:

- **B / S suffix or word** — *B*ig (larger-magnitude operand) vs *S*mall: `expB, sigB, sigS`.
- **sign / zsign** — the result's sign in the general case, vs the sign to use specifically when the datapath result is an exact zero (the two differ for –0 + –0 vs x + (–x)).
- **sub** — whether this addition is an effective subtraction (operand signs differ).
- **d12 / d21** — the exponent difference computed both ways (op1–op2 and op2–op1) in parallel, before it's known which operand is larger; the trailing digits name the direction, matching the subscript order.
- **ks** — normalisation *k*-shift: the left-shift amount applied in stage 3.
- **nan / inf / infSign / invalid** — special-case tags threaded through every stage, named directly after the IEEE class or flag they represent.
- **Stage1 / Stage2** — pipeline-register struct types, numbered by which stage produces them (consumed one stage later).

This mirrors the module's own doc comment, which lays out the pipeline as S1/S2/S3 and describes each stage's signals in the same terms.